Agora: One Sentence, One Executable World

A Typed Multimodal Compiler for Persistent Social Simulations

Boxun Xu Yueze Wang Peng Li

Abstract

Generating a world from language is often evaluated as text generation, image generation, or agent dialogue. None alone establishes that the result is an *executable world*: named entities must retain identity across semantics, media, geometry, persistent state, and a live client. We present AGORA, a project guided by the vision *One Sentence, One Executable World*. Its typed compiler expands a premise into specialist-owned world intermediate representations (IRs), generates world-conditioned FLUX assets, packages explicit state, and runs render-conditioned verification and localized repair. Provider, renderer, authoring, and live-runtime ports remain independently replaceable. We compare AGORA with a strong one-shot baseline on ten paired premises using the same Gemini model and temperature; the baseline's single call receives a 24,576-token cap, three times the specialist maximum. The specialist compiler satisfies the complete contract in 10/10 worlds versus 7/10 for the monolithic baseline ($p = 0.125$); first-pass completion is 9/10 versus 5/10 ($p = 0.109$). The baseline is nevertheless 1.8–1.9× faster and roughly 2× cheaper in tokens, establishing a distinct reliability–cost operating point. A frozen deterministic audit finds stronger wardrobe-to-canon grounding for decomposition on all five held-out worlds ($p = 0.031$), while monolithic generation directionally preserves more premise terms in agent text. After seeded role-contract faults, dependency routing repairs 5/5 worlds while preserving five upstream IR nodes, reducing provider calls by 48.0% and tokens by 39.9% versus full replay (token reduction $p = 0.031$). Separately, production validators detect and attribute 58/58 multimodal defects with 0/58 false positives. Five complete FLUX worlds pass compiler, media, browser, persistence, and publication gates; 20 ten-user runtime sweeps yield mean p95 latencies of 12.0 ms for state reads and 51.5 ms for movement. The result is an objective systems claim: typed decomposition purchases executable integrity and bounded repair at measurable inference cost.

1 Introduction

Consider the premise: "A municipal archive licenses gravity vectors as property." A generated illustration can depict the archive, and an LLM can role-play its clerks. A persistent interactive world requires more. Gravity licenses must be items owned by particular agents; counterfeit vectors must cause disputes; rooms must have walkable geometry; characters and machines must render with the intended identities; trades and messages must survive a client refresh. The output is not one sample. It is a versioned program with cross-modal invariants.

This distinction is easy to lose in text-to-game systems. A plausible monolithic JSON response may omit every character inventory. A valid image may belong to another room or revision. A component can be recorded as "placed" while being scaled to an unreadable footprint. A backend can pass a package smoke test while the browser fails to load an atlas. Conversely, external LLM latency can fail even when deterministic movement, persistence, and trade are correct. Treating these events as one success bit both hides defects and causes expensive full-world retries.

We formulate language-to-world generation as *typed multimodal compilation*. Specialist nodes expand the premise into semantic, visual, spatial, and state contracts. A compiler materializes explicit artifacts and provenance. Deterministic certificates establish facts such as dimensions, component coverage, and persistence; vision-language reviewers judge only failures that cannot be decided from the IR. Structured error codes route a failure back to its owning room, prop, or character.

AGORA is the project name. *One Sentence, One Executable World* states its authoring vision and executable-world contract.

This paper makes four contributions:

- A one-sentence-to-world task definition in which success requires an explicit, persistent, executable world rather than a generated frame.
- The AGORA specialist compiler, which preserves typed entity and revision identity across world semantics, FLUX media, geometry, package records, and a multiplayer client.
- A compiler-backed render-and-verify loop with substitution-free strict mode, measurable component readability, evidence-bound visual QA, cached asset certificates, and localized repair.
- A reproducible objective evaluation combining ten paired premise conditions, a strong monolithic baseline, frozen cross-modal measures, controlled local-repair and 58-defect ablations, five complete multimodal artifacts, and persistent-runtime latency.

2 Related Work

Procedural and language-driven game generation. Procedural content generation traditionally searches or constructs content under game-specific representations and constraints [1, 2]. MarioGPT maps prompts to controllable tile levels [3]. Whole-game and world authoring appears in Word2World, DreamGarden, STORY2GAME, and GameGPT [4, 5, 6, 7]; MetaGPT and ChatDev provide broader evidence for role-specialized software workflows [8, 9]. AGORA focuses on the resulting persistent artifact: typed cross-modal identity, state, and executable validation.

Interactive world models. Genie [10] and GameGen-X [11] generate interactive visual experience through learned world or video models. Their representation differs from a database-backed simulation with explicit agents, ownership, knowledge, messages, and revisioned assets. We therefore evaluate our executable contract rather than use video fidelity as a cross-task baseline.

Persistent agents and controlled media. Generative Agents and SOTOPIA study persistent social behavior and goals [12, 13]; Voyager couples executable skills with environment feedback [14]; and Concordia supports grounded generative agent-based models [15]. For media, scene graphs expose object relations [16]; ControlNet, GLIGEN, LayoutDiffusion, and IP-Adapter add structural, layout, and reference conditioning [17, 18, 19, 20]; and StoryDiffusion and Sprite Sheet Diffusion target cross-image and animation consistency [21, 22]. AGORA binds these concerns through authoritative state, typed visual policy, procedural geometry, and composed-client checks.

Structured generation and repair. Grammar-constrained decoding can guarantee syntactic form for structured NLP outputs [23], while Self-Refine improves model outputs through iterative feedback [24]; Reflexion similarly learns from environmental feedback without weight updates [25]. AGORA additionally validates cross-entity identity, geometry, provenance, media readability, and runtime behavior, then routes typed failures to their owning nodes.

3 Task Definition

Under the AGORA vision, a natural-language premise p is compiled to

$$W = (E, R, G, S, A, P),$$

where E contains entities such as rooms, agents, items, and institutions; R contains typed social and economic relations; G contains geometry and collision state; S is mutable persistent state; A contains media; and P contains package and provenance records. A world is executable only if it satisfies four contracts:

1. **Semantic validity:** all required typed fields and cross-entity references compile without synthetic substitution.
2. **Referential integrity:** every generated asset is bound to an entity, world revision, and generation condition.
3. **Spatial and perceptual validity:** geometry matches renderer dimensions; required media are present, readable, and projection compatible.
4. **Runtime validity:** the package starts and a browser can move, persist a message, use and transfer an item, quote a trade, and settle it.

For asset a intended for entity e, cross-modal referential integrity is a total mapping

$$\pi(a) = (e, v, c, h),$$

where v is the world revision, c is the typed condition derived from the IR, and h is a content or certificate fingerprint. This mapping prevents a valid but stale or unrelated asset from satisfying the current world.

"One sentence" is the author-supplied semantic condition. A fixed evaluator-owned contract controls scale, semantic-prop coverage, camera, and forbidden rendering artifacts. It is task protocol, not additional world content: premises change while executable requirements do not, and absent typed state cannot be replaced by generic content.

4 Agora Architecture

4.1 Specialist-Owned Typed IR

Figure 1 summarizes AGORA. A planner establishes world identity, theme, institutions, scarce resources, and gameplay loops. Separate nodes own materials, rooms, items, roles, agents, visual canon, wardrobe, properties, and knowledge. Each node receives a bounded context and returns schema-constrained JSON. The normalized builder specification is the sole source for scenario configuration and package compilation.

Decomposition does more than reduce output length. It assigns every invariant an owner. An empty inventory is repaired by the character or item node; a missing doorway belongs to spatial compilation; incompatible clothing belongs to wardrobe policy. No node may silently inject generic ledgers, notices, or coupons to make an invalid character compile. Strict mode stops at the violating node and preserves the invalid response as evidence.

4.2 Semantic Media and Geometry

A generated visual canon records camera, palette, materials, lighting, silhouettes, and forbidden motifs. A separate wardrobe policy maps social roles to clothing and

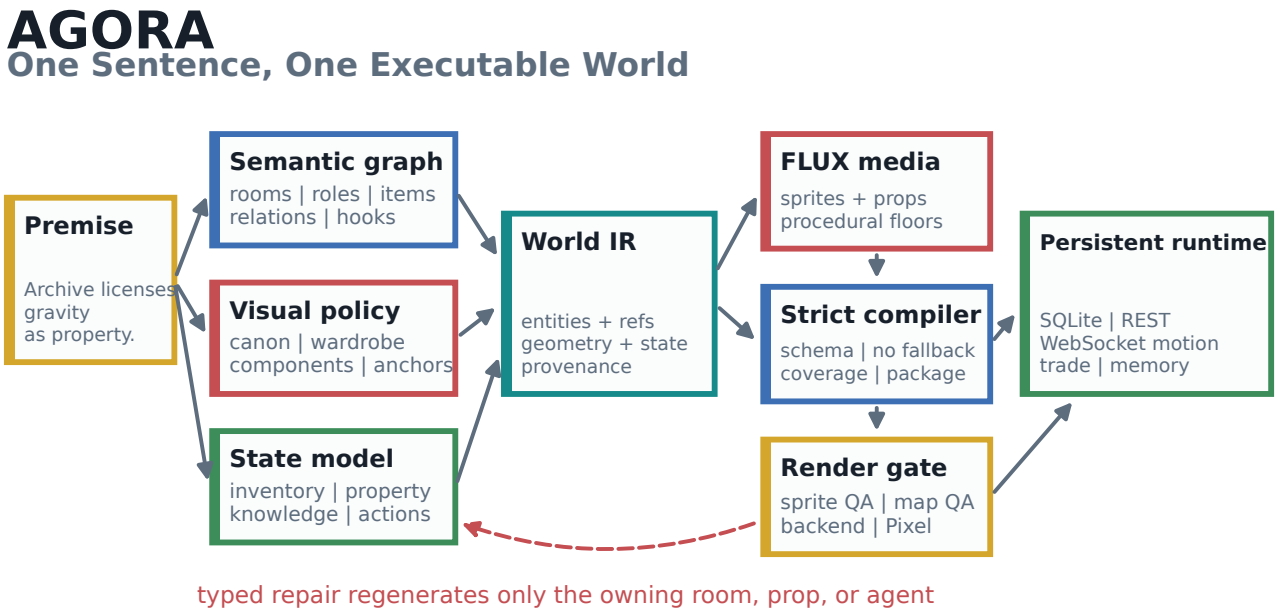

Figure 1: AGORA architecture. The vision *One Sentence, One Executable World* defines the one-premise-to-executable-world objective. The compiler carries that contract through typed IR, media, package state, and runtime verification; structured failures return to the smallest owning node. Live dialogue latency is measured separately from the deterministic publication gate.

carried tools. Room IR includes purpose, visual components, component anchors, and tile footprint. Character and component prompts are derived from these records, not from a global hard-coded setting.

The map branch deliberately separates geometry from semantic media. Procedural layered floor plates obey exact tile dimensions and collision coordinates. FLUX generates premise-specific semantic components, while a component compiler removes studio backgrounds, normalizes assets, and places them at typed anchors. The character branch generates directional pixel sheets, converts them to Phaser atlases, and records agent, world, and revision identity in each bundle.

4.3 Certificates and Localized Repair

The publication gate combines deterministic checks with visual review. Deterministic checks own questions with exact answers: file presence, map dimensions, alpha bounds, sprite frame integrity, expected component IDs, component footprint, collision, package startup, and browser persistence. Vision-language QA owns residual visual defects such as baked characters, incompatible perspective, visible text, or palette drift.

Component placement sidecars were upgraded from "paste succeeded" to a v2 readability certificate. For each semantic component, the compiler records its rendered size, room-footprint ratio, effective opaque ratio, and pass bit. Square FLUX icon canvases receive a minimum readable footprint even when the authored object is long and thin. Collision-aware anchor search prefers the authored anchor and deterministically relocates colliding components.

VLM error codes require textual evidence. Projection errors, for example, must mention a side face, front face, horizon, projection, or lost footprint. The reviewer cannot overrule compiler-established presence by asserting that a certified component is missing; it can still fail that room by stating the observable defect, such as a tiny prop cluster leaving a large empty floor. Each verdict is appended to revision history. A room-local error regenerates only that room or its components, preserving accepted floors and characters.

Sprite-batch visual certificates are keyed by a SHA-256 fingerprint over the QA contract, visual canon, agent IDs, and raw sheets. Reused assets with an identical fingerprint skip another external review. A changed sheet or canon invalidates the certificate. This turns transient provider failure into a retryable review problem rather than corruption of already-certified media.

4.4 Package and Persistent Runtime

The compiler writes scenario JSON plus a SQLite `world_package.db`. Publication materializes only revision-matching assets into an isolated package. The Pixel client loads authoritative state, creates DB-backed sessions, claims agents, reads state over REST, and sends

movement over WebSocket. Item use, transfer, quote, settlement, messages, and memories are written back to the database.

The headless publication profile validates these deterministic actions without waiting for an external model response. A separate full-reply profile measures provider and visible response latency. This split prevents a temporary model 503 or DNS failure from being mislabeled as a broken package, while retaining an explicit latency benchmark for live interaction.

4.5 Scalability and Extensibility

AGORA scales through explicit service boundaries rather than a single model-specific call path. The structured-generation port supports JSON, text, media-conditioned, and streamed-field requests. Configuration selects AI Studio REST or Vertex Publisher REST and independently sets model, output cap, thinking policy, timeout, and retry policy per specialist stage. Provider failover reissues the same typed request through the alternate route; the response still faces the same schema and compiler, so transport recovery never authorizes generic world content.

Prompt synthesis is likewise separated from rasterization. Character, room, item, and component jobs carry prompts, dimensions, seeds, sampling controls, and optional conditioning images across an HTTP image port. The evaluated deployment uses the FLUX `/generate` service. An SD-WebUI `txt2img` transport is also implemented; direct Gemini/Vertex raster generation is intentionally disabled. Unknown renderers fail at dispatch rather than silently selecting another backend.

Authoring requests append revision-addressed artifact state and queue generation and art as detached workers. Character assets expose a configurable worker pool, while renderer endpoints can be deployed independently of the compiler. Published SQLite packages and materialized static assets decouple world delivery from authoring. At runtime, infrequent state and transactional actions use REST, while movement uses a dedicated WebSocket loop. These boundaries permit independent replication and back-pressure policies; Table 1 distinguishes implemented surfaces from paths exercised in this paper.

Layer	Implemented ports	Evaluated
Structured models	AI Studio and Vertex Publisher REST; JSON, text, media, stream	AI Studio
Raster media	FLUX `/generate`; SD-WebUI `txt2img`	FLUX
Model gateway	Flex execute; OpenAI-compatible chat endpoint	No
Authoring	Draft, revise, art, status, package, publish REST	Yes
Live world	Package/state/action REST; movement WebSocket	Yes
Delivery	Revision DB, access code, materialized static assets	Yes

Table 1: Implemented integration surfaces. "No" denotes an extension port, not an evaluated performance claim.

Set	N	Purpose
Primary generation	15/treat.	Repeated completion and cost
Held-out generation	5/treat.	New world clusters and quality
Full multimodal audit	5 worlds	Media, provenance, browser, publish
Seeded local repair	5 worlds	Dependency-aware regeneration
Paired validator faults	58	Detection, attribution, false positives
Runtime	10 worlds	Ten-user REST and WebSocket paths

Table 2: Trials are not treated as independent worlds.

5 Evaluation

5.1 Artifact Sets

Table 2 separates datasets by question. Five primary premises receive three generation trials per treatment. Five frozen held-out premises, which predate the experiment and retain their original generation requests and complete media, receive one trial per treatment. Together they form ten independent world clusters for paired reliability and deterministic quality analysis. Only the five primary worlds are used for the fully revalidated multimodal artifact audit; runtime sweeps use ten complete published worlds and do not estimate generation quality.

5.2 Metrics and Protocols

The strict audit checks compiler status, package and map presence, prohibited substitute IDs, semantic component generation, sidecar-certified placement, sprite-batch QA, map QA, backend startup, headless Pixel launch, and publication readiness.

Both generation treatments use `gemini-3-flash-preview`, temperature 0.2, and low thinking. The specialist treatment runs the production planner, visual-canon, rooms, materials, items, batched roles, wardrobe, hooks, and summary nodes with stage-specific caps up to 8192 tokens per call. The monolithic treatment makes one call for the complete merged builder specification and receives a 24576-token cap. Its prompt explicitly requires distinct role niches, cross-character relations, and at least 15 inventory entries for every merchant-like character. Giving the one-shot baseline three times the output capacity removes a simple truncation explanation and favors it on wall time and call count.

Both outputs face the same schema, substitution-free merged contract, and deterministic compiler. Success requires valid JSON, all semantic and multimodal fields, complete merchant inventories, and compilation. We report first-pass and eventual success separately. A primary world uses majority outcome across its three replicates; a held-out world uses its single frozen trial. The confir-

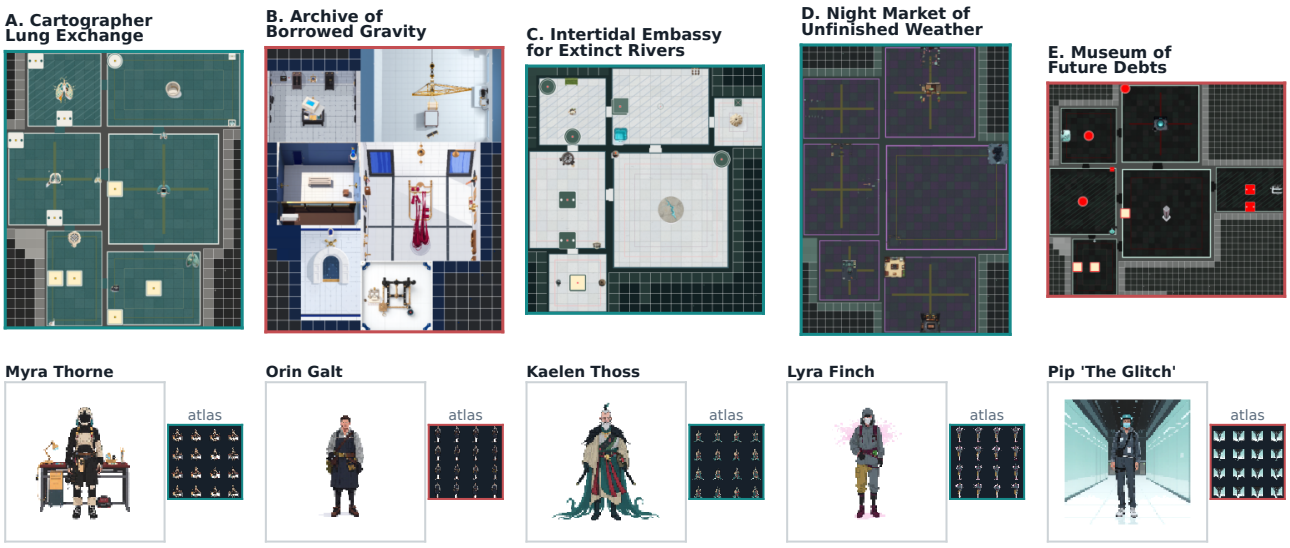

Figure 2: Five strict multimodal artifacts compiled by the same pipeline: (A) Cartographer Lung Exchange, (B) Archive of Borrowed Gravity, (C) Intertidal Embassy for Extinct Rivers, (D) Night Market of Unfinished Weather, and (E) Museum of Future Debts. Each contains six rooms and twelve agents and passes the final compiler, backend, headless Pixel, map-QA, and publication gate.

matory analysis unit is therefore the world, followed by an exact one-sided paired sign test. Provider attempts, wall time, finish reason, and usage metadata are recorded for every call without storing prompts or credentials in telemetry.

No model output is removed because it fails the contract, and all completed trials appear in the per-world matrix. One pre-experiment sandbox launch could not resolve the provider endpoint and returned no model response; it was voided before the clean run rather than counted as a generation trial. The five held-out requests and deterministic evaluator were frozen before held-out generation.

The deterministic quality evaluator measures home-base and room coverage, role and inventory uniqueness, merchant inventory compliance, lexical premise recall by semantic branch, explicit cross-character relations, and room/wardrobe adherence to generated visual canon. No LLM judge is used. We report unadjusted paired sign tests as diagnostic measures and separately identify the five-world held-out result.

For the local-repair ablation, the evaluator injects a role-inventory contract defect after the first complete specialist-node pass. The production retry router receives the resulting typed error. We compare the calls and tokens in the regenerated dependency subgraph with replaying the complete initial node set; all provider usage is measured rather than estimated.

The seeded-fault benchmark contains twelve defect types in five ownership groups: semantic IR, asset provenance, sprite source/atlas, map dimensions, and semantic-component placement. It modifies temporary copies only and calls production deterministic validators, with no model or image-generation requests. We report detection, exact error attribution, paired clean-control false positives, and Wilson 95% intervals.

The runtime protocol executes one ten-user REST sweep and one ten-user WebSocket sweep for each of ten worlds, for 20 runs total. It measures live state, heartbeat, connection, and movement without LLM actions. A separate five-world browser probe records message persistence, provider latency, and visible reply latency.

6 Results

6.1 One-Sentence Premise Benchmark

The five conditions begin with one creative sentence defining a world: breathable atmosphere traded inside an orbital lung; gravity vectors licensed as municipal property; extinct rivers negotiating legal personhood; unfinished weather sold before release; and future debts exhibited before they exist. The evaluator then applies the same executable-world contract: six functional rooms, twelve interdependent agents, world-specific semantic components, strict top-down rendering, and no visible text or unrelated style leakage. This protocol isolates premise variation while making success machine checkable. Every resulting world reaches the publication gate; none requires generic inventory substitution.

6.2 Specialist Versus Strong Monolithic Generation

Across ten paired worlds, specialist generation eventually satisfies the complete contract in 10/10 cases ver-

Figure 3: Raw paired evidence rather than selected examples. (A) First-pass and eventual outcomes for all ten worlds; primary outcomes use majority over three replicates. (B) Reliability versus measured token and latency cost; marker area scales with median wall time. (C) Every world-level quality difference; diamonds are means and displayed tests are unadjusted. (D) All five seeded repair traces; calls and tokens compare with full replay, time with the measured initial pass, and the dashed line marks parity.

sus 7/10 for the strong monolithic baseline (Figure 3A). All three discordant worlds favor the specialist treatment, giving a one-sided exact $p = 0.125$. First-pass world-level completion is 9/10 versus 5/10; five discordant worlds favor specialists and one favors the monolith ($p = 0.109$). These world-level results are directionally consistent but do not establish a significant general reliability advantage at $n = 10$.

The three-replicate primary set exposes repeatability without changing the analysis unit. Specialist generation completes 15/15 trials, 14 on the first pass. Monolithic generation completes 9/15, all on the first pass; the six other outputs are valid JSON and compile, but fail the explicit merchant inventory contract. A trial-level paired calculation gives $p = 0.0156$, but replicates share premises and are not independent evidence. On the held-out set, specialists complete 5/5 (4 first-pass), while the monolith completes 4/5 (2 first-pass); two monolithic outputs recover from token-limit truncation and one compiled output again omits required merchant inventory.

Split	Method	Strict	First	Tok.	Med. s
Primary	Specialist	15/15	14/15	48.5k	101.2
Primary	Monolithic	9/15	9/15	22.2k	55.8
Held-out	Specialist	5/5	4/5	77.6k	161.1
Held-out	Monolithic	4/5	2/5	38.9k	86.1

Table 3: Measured generation cost per trial. Held-out worlds contain 24–25 agents versus 12 in the primary set.

Table 3 and Figure 3B make the cost explicit. On the primary set, specialists consume 2.18× the tokens and 1.82× the median wall time; on held-out worlds the factors are 1.99× and 1.87×. The monolith therefore remains the efficient choice when its output happens to satisfy all constraints. Typed decomposition buys more reliable constraint completion and repair ownership, not free generation.

Fault owner	N	Detect	Attrib.	FP
Semantic IR	25	25	25	0/25
Asset provenance	10	10	10	0/10
Sprite source/atlas	10	10	10	0/10
Map dimensions	5	5	5	0/5
Component placement	8	8	8	0/8
Total	58	58	58	0/58

Table 4: Seeded defects use production deterministic validators. Each mutation has a matched clean control.

6.3 Frozen Objective Quality

Figure 3C compares deterministic measures over one paired output per world. Wardrobe descriptions cover 50.7% of generated canon terms under decomposition versus 32.1% for the monolith; specialists win 9/10 worlds (exploratory $p = 0.0107$). More importantly, the independently generated held-out subset preserves the effect: 47.7% versus 21.2%, with specialists winning 5/5 ($p = 0.031$). This supports the architectural claim that a separately owned wardrobe node can carry a visual contract into every character.

Other measures are mixed. Premise-item recall is 34.8% versus 23.3% (8/10 specialist wins, $p = 0.0547$); merchant compliance is 100% versus 82.5% (four wins and six ties, $p = 0.0625$); and room-to-canon adherence is 90.0% versus 81.3% ($p = 0.109$). Conversely, the monolith retains more premise terms in agent descriptions (55.9% versus 46.3%; seven monolithic wins, two specialist wins, one tie). Macro premise recall is effectively tied (35.6% versus 35.7%). These lexical measures do not establish perceptual quality, and their unadjusted tests are reported as mechanism diagnostics rather than a composite state-of-the-art score.

6.4 Seeded-Fault Detection and Attribution

Table 4 evaluates the verifier independently of generation quality. Across 58 mutations, production validators detect 58/58 and return the expected owning error code for 58/58. The 58 paired clean controls produce no false positives. Aggregate detection and attribution are 100% with Wilson 95% CI 93.8–100%; clean specificity has the same interval. The per-class intervals are necessarily wider because each class contains only five worlds, or four for placement-sidecar faults.

6.5 Strict Multimodal Integrity

The five strict worlds contain 30 rooms, 60 agents, and 594 protagonist inventory entries. All five compile and pass the publication gate, with zero prohibited generic substitutions. The media pipeline generates all 48 expected semantic components. Four worlds have placement sidecars and certify 38/38 expected placements; Archive of Borrowed Gravity predates the sidecar and is reported only as 10/10 generated component artifacts. We do not promote generation into unobserved placement evidence.

Strict audit measure	Result
Compiler / publication gate	5/5; 5/5
Backend startup / Pixel launch	5/5; 5/5
Map QA final pass	5/5
Sprite-batch QA	60/60
Semantic components generated	48/48
Post-sidecar placements	38/38
Generic inventory substitutions	0/594

Table 5: Machine-generated strict multimodal audit.

All 60 character sheets pass batch vision QA, and all five maps finish with no actionable map-QA code. Backend startup and Pixel launch pass in 5/5 cases. The headless client verifies rendering and movement; recent deterministic profiles additionally exercise message persistence, self item use, target item transfer, quote creation, and settlement without requiring an external reply.

6.6 Placement-Compiler Ablation

Night Market of Unfinished Weather exposes why referential coverage alone is insufficient. The v1 sidecar recorded all ten semantic components as pasted, yet destination boxes inherited skinny 3×1 or 1×3 footprints for square FLUX canvases. Across the same ten component IDs, the minimum and median short edge were 24 and 54 pixels. The v2 placement compiler sets a readable square-aware lower bound and records opacity and room coverage; minimum and median short edges become 86 and 86 pixels, with 10/10 readability certificates and a passing Pixel gate.

This is an objective layout ablation, not an aesthetic score. It shows that "asset exists" and "asset is visibly instantiated" are different contracts, and that the latter can be checked before consulting a VLM.

6.7 Seeded Localized Generation Repair

Figure 3D measures the implemented generation-repair route, not an offline simulation. In each primary world, the evaluator injects a role-inventory violation after the first complete node pass. Production error routing identifies the roles branch in 5/5 cases, invalidates roles, wardrobe, and hooks, and preserves planner, visual canon, rooms, materials, and items. All five repaired specifications then satisfy the merged contract and compile.

Relative to replaying the complete initial node set, localized repair uses 48.0% fewer provider calls and 39.9% fewer tokens on average. Token use is lower in 5/5 worlds (one-sided sign test $p = 0.031$). Median measured provider time is 52.1 s for the repair subgraph versus 85.7 s for the initial pass; time is lower in 4/5 worlds ($p = 0.188$); call and token reduction provide the statistically supported efficiency result. One world also exhibits a natural pre-seed role-contract retry, which is attributed separately and excluded from the nominal initial node set.

Thus the savings are measured from real calls while the seeded intervention controls fault ownership.

6.8 Runtime Scalability

All 20 ten-user sweeps, each issuing concurrent client traffic, complete without backend stress failures. Mean REST live-state p50/p95 is 5.0/12.0 ms; heartbeat is 4.5/5.7 ms; and WebSocket movement delta is 28.5/51.5 ms. The maximum per-world p95 is 22.7 ms for state and 51.9 ms for movement. WebSocket connection setup is slower (604.5 ms mean p95) but is not on the movement hot path.

The five-world full-reply probe succeeds in 5/5 cases. Visible reply latency is 3148.6 ms mean and 3539.8 ms p95, including provider latency of 1671.0/1780.8 ms mean/p95. Message persistence in that profile is 995.6/1190.4 ms. External inference latency must not be used as a proxy for persistent-state or movement performance.

7 Discussion

The differentiator is explicit execution. AGORA offers an advantage orthogonal to frame fidelity: a concise premise becomes an inspectable program whose entities, relations, geometry, media provenance, mutable state, and client actions survive packaging and replay. This representation supports tests unavailable to frame-only systems, including ownership, inventory transfer, settlement, and revision identity.

Compiler evidence should bound learned review. VLM reviewers are useful for projection and composition but can contradict observable facts. During repair, reviewers sometimes described certified components as absent. Evidence-bound QA resolves this without blindly accepting the map: the compiler decides presence and readable footprint; the reviewer may still report sparse composition or incompatibility using the corresponding visual code. This division converts a stochastic judge from an oracle into one typed analysis pass.

Decomposition enables economical repair. A world contains many accepted artifacts. Regenerating all of them after one room error changes unrelated identities and multiplies inference cost. Specialist ownership, revisioned provenance, and localized error codes allow the system to preserve floors, characters, or components outside the failure scope. Fingerprinted QA certificates similarly avoid paying a remote reviewer to reconfirm byte-identical sprite sheets. The 58 seeded validator faults establish exact ownership under controlled artifact mutations. The five end-to-end role interventions show the complementary operational result: production routing preserves five completed IR nodes and reduces replayed calls and tokens while still compiling every repaired world.

Agora defines a reliability–cost frontier. The monolith is approximately twice as token-efficient and faster, but loses the complete executable contract in three of ten worlds. AGORA satisfies all ten, strengthens wardrobe-to-canon grounding, and preserves typed repair ownership across every accepted artifact. Decomposition therefore buys end-to-end contract reliability, cross-modal traceability, and bounded repair at a measurable inference cost. This is the operating point required when a generated world must be published and replayed rather than merely sampled.

8 Conclusion

We presented AGORA, guided by *One Sentence, One Executable World*, as a typed multimodal world compiler. Across ten premises, AGORA turns one sentence into complete executable contracts more reliably than a strong monolith. Its typed ownership delivers exact fault attribution and localized repair with fewer calls and tokens than full replay. Five complete multimodal worlds and ten-user sweeps demonstrate that the resulting programs execute, persist, render, and move. AGORA therefore advances language-to-world generation from plausible samples to versioned worlds whose entities, media, state, faults, and actions are jointly testable.

References

[1] N. Shaker, J. Togelius, and M. J. Nelson. *Procedural Content Generation in Games*. Springer, 2016.

[2] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne. Search-based procedural content generation: A taxonomy and survey. *IEEE TCIAIG*, 2011.

[3] S. Sudhakaran, M. González-Duque, C. Glanois, et al. MarioGPT: Open-ended text2level generation through large language models. *NeurIPS*, 2023.

[4] M. U. Nasir, S. James, and J. Togelius. Word2World: Generating stories and worlds through large language models. *arXiv:2405.06686*, 2024.

[5] S. Earle, S. Parajuli, and A. Banburski-Fahey. DreamGarden: A designer assistant for growing games from a single prompt. *arXiv:2410.01791*, 2024.

[6] E. Zhou, S. Basavatia, M. Siam, Z. Chen, and M. O. Riedl. STORY2GAME: Generating (almost) everything in an interactive fiction game. *arXiv:2505.03547*, 2025.

[7] D. Chen, H. Wang, Y. Huo, Y. Li, and H. Zhang. GameGPT: Multi-agent collaborative framework for game development. *arXiv:2310.08067*, 2023.

[8] S. Hong, M. Zhuge, J. Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *ICLR*, 2024.

[9] C. Qian, W. Liu, H. Liu, et al. ChatDev: Communicative agents for software development. *ACL*, 2024.

[10] J. Bruce et al. Genie: Generative interactive environments. *ICML*, 2024.

[11] H. Che, X. He, Q. Liu, C. Jin, and H. Chen. GameGen-X: Interactive open-world game video generation. *ICLR*, 2025.

[12] J. S. Park et al. Generative agents: Interactive simulacra of human behavior. *UIST*, 2023.

[13] X. Zhou, H. Zhu, L. Mathur, et al. SOTOPIA: Interactive evaluation for social intelligence in language agents. *ICLR*, 2024.

[14] G. Wang, Y. Xie, Y. Jiang, et al. Voyager: An open-ended embodied agent with large language models. *TMLR*, 2024.

[15] A. S. Vezhnevets, J. P. Agapiou, A. Aharon, et al. Generative agent-based modeling with actions grounded in physical, social, or digital space using Concordia. *arXiv:2312.03664*, 2023.

[16] J. Johnson, A. Gupta, and L. Fei-Fei. Image generation from scene graphs. *CVPR*, 2018.

[17] L. Zhang and M. Agrawala. Adding conditional control to text-to-image diffusion models. *ICCV*, 2023.

[18] Y. Li, H. Liu, Q. Wu, et al. GLIGEN: Open-set grounded text-to-image generation. *CVPR*, 2023.

[19] G. Zheng, X. Zhou, X. Li, et al. LayoutDiffusion: Controllable diffusion model for layout-to-image generation. *CVPR*, 2023.

[20] H. Ye, J. Zhang, S. Liu, X. Han, and W. Yang. IP-Adapter: Text compatible image prompt adapter for text-to-image diffusion models. *arXiv:2308.06721*, 2023.

[21] Y. Zhou, D. Zhou, M.-M. Cheng, J. Feng, and Q. Hou. StoryDiffusion: Consistent self-attention for long-range image and video generation. *NeurIPS*, 2024.

[22] C.-A. Hsieh, J. Zhang, and A. Yan. Sprite Sheet Diffusion: Generate game character for animation. *arXiv:2412.03685*, 2024.

[23] S. Geng, M. Josifoski, M. Peyrard, and R. West. Grammar-constrained decoding for structured NLP tasks without fine-tuning. *EMNLP*, 2023.

[24] A. Madaan, N. Tandon, P. Gupta, et al. Self-Refine: Iterative refinement with self-feedback. *NeurIPS*, 2023.

[25] N. Shinn, F. Cassano, E. Berman, et al. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.